Lab6 - Building a Router and Dive into TCP

200 points

Lab Environment:

- Part 1 and Part2 of this lab have been tested on the VM image in the lab(s). We cannot provide support for other environments – some things might work and others not etc. If you have an M1/M2 Mac which does not work properly or are otherwise unable to run the VM, please use the lab computers to complete the assignment. Also, make sure your scripts are written Python 3.

Helpful Resources:

- Computer Networks: A Top Down Approach - Chapter 3 covers TCP
- **POX Wiki**
- Inside your VM, the /pox/pox/forwarding/l2_learning.py example file
- UDP:
  - Brief UDP Summary
  - RFC 768
- TCP Stevens Graph: helpful video!

Submission:

- **cruzid-lab6.pdf** : PDF containing answers to all questions, submitted on Gradescope.
- **cruzid-lab6_topo.py** : Your network topology for the router submitted on Canvas.
- **cruzid-lab6_controller.py** : Your remote controller code for the router submitted on Canvas.

Due Dates:

- Lab6 Part 1 Due:11/10, Sunday 11:59 pm
  - Last Day to Submit: 11/15, Friday 11:59 pm
- Lab6 Part 2 Due: 11/16, Saturday 11:59 pm
  - Last Day to Submit: 11/22, Friday 11:59 pm

Lab 6 Part 1 Demo:

- Something NEW! Practice Your Speaking! For Lab 6 part 1 during Week 7(11/11-11/15), there will be **30** points for an in-person demo. All students must attend during their **assigned lab section** where you will have a discussion about your code with someone from the Teaching Team.

Introduction: In Lab5, we learned about Software Defined Networking (SDN) and the POX Controller to design a Firewall. In the first part of this Lab, we are going to use the POX Controller to build a router. This lab again assumes you will spend time reading the documentation and learning about Mininet, OpenFlow and the POX controller.
In the second half of the assignment, we explore the internal workings of the TCP protocol.

Screenshot Table					
Part 1	**Q1**	**Q2**	**Q3**	**Q4**	**Q5**
Part 2	**Q6ab**	**7ab**	**8b**	**9**	**10**
	11b	**12b**	**13ac**		

[100 pts] Part 1: Router (5 pts progress check in lab)

In this assignment we will implement a firewall for the Enterprise Network topology shown in Figure 1. **Note the departments are organized into subnets.** Your goal will be to allow or block traffic between the different devices based on the 4 Network Topology Rules given below. In this assignment, you **cannot** use flooding **unless you are doing ARP**(i.e., you cannot use flooding: `of.OFPP_FLOOD or of.OFPP_NORMAL`)**!** Instead, you will need to specify specific ports for all traffic and the forwarding must be done by subnet, not by enumerating all of the IP addresses**.** You might consider a method to determine if an IP Address is valid on a particular subnet. Additionally, notice that the rules are written according to the subnet.

You may implement your router however you choose—although as a suggestion, you may find it easiest to determine the correct destination port by using either

- the source and destination IP addresses
- or the source port on the switch from which the packet originated.

These provided files will get you started—you will need to modify both:
`lab6_topo_skel.py` / `lab6_controller_skel.py`

. You will need to do the following:

- Complete the topology file by adding all the hosts (be sure to manually specify the MAC address, IP address and subnet for each host), switches and links
- Implement the router based on the 4 Network Topology Rules

4 Network Topology and Rules to be implemented:

- **Rule 1:** `ICMP` traffic is forwarded only between the Student Housing LAN, Faculty LAN and IT Department subnets or between devices that are on the same subnet.
- **Rule 2:**`TCP` traffic is forwarded only between the University Data Center, IT Department, Faculty LAN, Student Housing LAN**, trustedPC**, or between devices that are on the same subnet; however, **only the Faculty LAN may access the exam server**.
- **Rule 3:** `UDP` traffic is forwarded only between the University Data Center, IT Department, Faculty LAN, Student Housing LAN, or between devices that are on the same subnet
- **Rule 4:** All other traffic should be dropped.

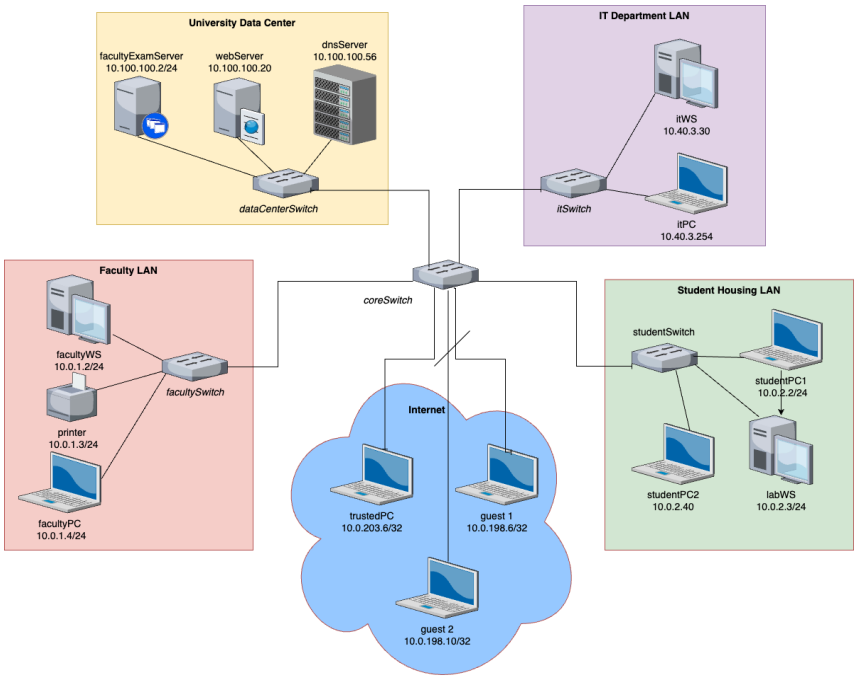

Figure 1

Your code must use the required naming convention for the switches and hosts in Table 1:

Device	Required Naming Convention					
	Core	**Faculty LAN**	**Student Housing LAN**	**IT Department LAN**	**University Data Center**	**Internet**
Workstations		‘facultyWS’	‘labWS’	‘itWS’		
Laptops		‘facultyPC’	‘studentPC1’ ‘studentPC2’	‘itPC’		‘trustedPC’, ‘guest1’ ‘guest2’
Servers					‘examServer’, ‘webServer’, ‘dnsServer’	
Other		‘printer’				
Switches	‘s1’	‘s2’	‘s3’	‘s4’	‘s5’	

Questions and Rule Verification:

Question 1 [5 pts]: Annotate the topology in Figure 1 with port numbers associated with the end of each link and include a **screenshot** of the detailed image of Figure 1. Then, implement the topology in the skeleton file lab6_topo_skel.py.

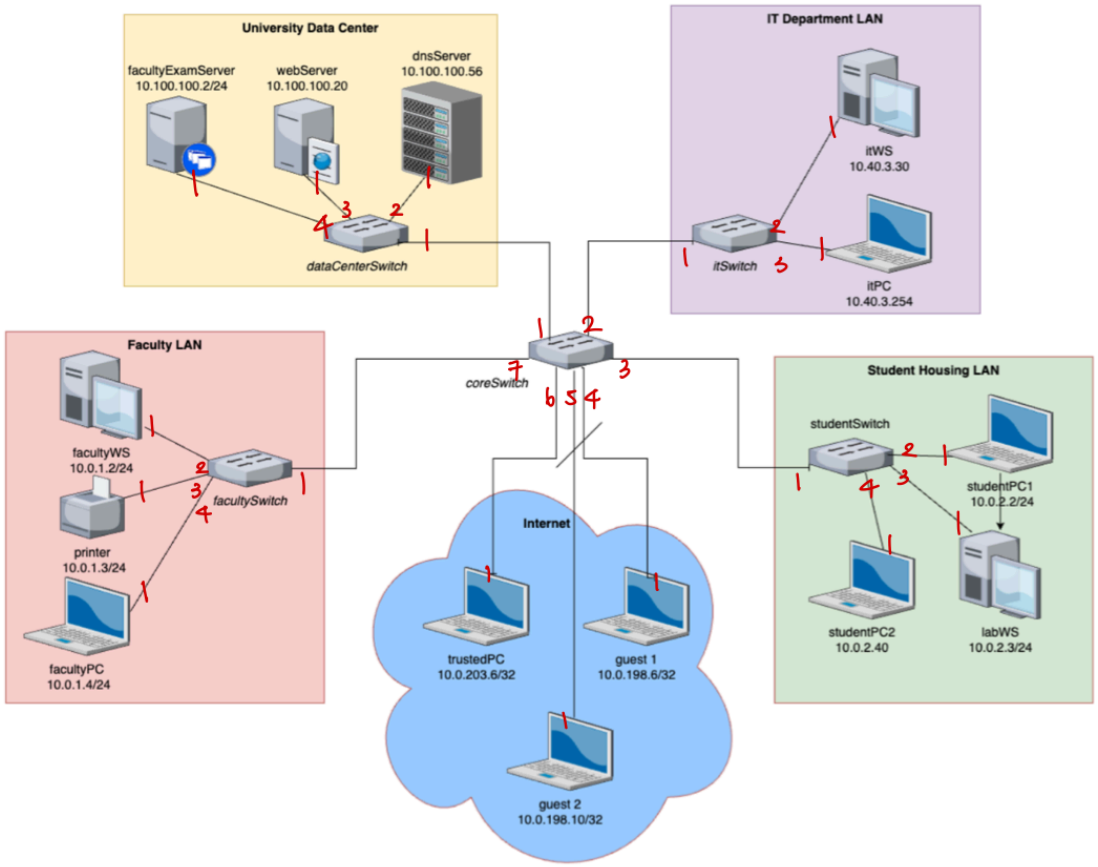

Figure 1

Explain how packets are forwarded differently in this assignment compared to Lab5 (think about your previous `accept()` function.)

Unlike lab 5, packets are no longer being multicasted, meaning packets are not being forwarded to every port in order to get to one destination. Each host has their own designated port through which the switch in the subnet will forward packets.

Question 2 [10 pts]: After you have finished writing your code:

Verify Rule 1 with `pingall`. Circle in red the lines of your output that indicate the inter-departmental communication is working as expected. Explain your answer. Include a **screenshot** and refer to it in your explanation.

```
mininet> pingall
*** Ping: testing ping reachability
dnsServer -> examServer X X X X X X X X X X X webServer
examServer -> dnsServer X X X X X X X X X X X webServer
facultyPC -> X X facultyWS X X itPC itWS labWS printer studentPC1 studentPC2 X X
facultyWS -> X X facultyPC X X itPC itWS labWS printer studentPC1 studentPC2 X X
guest1 -> X X X X guest2 X X X X X X X X
guest2 -> X X X X guest1 X X X X X X X X
itPC -> X X facultyPC facultyWS X X itWS labWS printer studentPC1 studentPC2 X X
itWS -> X X facultyPC facultyWS X X itPC labWS printer studentPC1 studentPC2 X X
labWS -> X X facultyPC facultyWS X X itPC itWS printer studentPC1 studentPC2 X X
printer -> X X facultyPC facultyWS X X itPC itWS labWS studentPC1 studentPC2 X X
studentPC1 -> X X facultyPC facultyWS X X itPC itWS labWS printer studentPC2 X X
studentPC2 -> X X facultyPC facultyWS X X itPC itWS labWS printer studentPC1 X X
trustedPC -> X X X X X X X X X X X X X
webServer -> dnsServer examServer X X X X X X X X X X X
*** Results: 64% dropped (64/182 received)
```

Yes this was expected. ICMP traffic is forwarded only between the Student Housing LAN, Faculty LAN and IT Department subnets or between devices that are on the same subnet. Hence, the servers can only contact each other, and the three subnets can communicate with each other.

<u>Question 3 [20 pts]:</u> Verify Rule 2 with `iperf`. Are the individual results what you expect? Why? Include a **screenshot** and refer to it in your explanation after running *iperf* between these pairs of nodes:

Note: I assume the guestPC is guest1

- trustedPC and guestPC

```
mininet> iperf trustedPC guest1
*** Iperf: testing TCP bandwidth between trustedPC and guest1
^C
Interrupt
```

It failed because trustedPC and guest1 are not in the same subnet, and hence TCP is blocked.

- guestPC and webServer

```
mininet> iperf guest1 webServer
*** Iperf: testing TCP bandwidth between guest1 and webServer
^C
Interrupt
```

Guest1 and webServer are not in the same subnet, and hence, the TCP is blocked.

- examServer and studentPC2

```
mininet> iperf examServer studentPC2
*** Iperf: testing TCP bandwidth between examServer and studentPC2
^C
Interrupt
```

The examServer is only accessible from the faculty LAN, and hence the TCP from examServer to studentPC2 is blocked.

- studentPC2 and facultyWS

```
mininet> iperf studentPC2 facultyWS
*** Iperf: testing TCP bandwidth between studentPC2 and facultyWS
*** Results: ['23.9 Gbits/sec', '24.0 Gbits/sec']
```

FacultyLAN and student housing LAN can communicate over TCP, and hence the connection goes through

- facultyWS, LabWS

```
mininet> iperf facultyWS labWS
*** Iperf: testing TCP bandwidth between facultyWS and labWS
*** Results: ['28.6 Gbits/sec', '28.7 Gbits/sec']
```

FacultyLAN and student housing LAN can communicate over TCP, and hence the connection goes through

- itWS to itPC

```
mininet> iperf itWS itPC
*** Iperf: testing TCP bandwidth between itWS and itPC
*** Results: ['31.1 Gbits/sec', '31.2 Gbits/sec']
```

Devices on the same subnet can communicate with each other, and hence the TCP connection goes through.

- trustedPC and itPC

```
mininet> iperf trustedPC itPC
*** Iperf: testing TCP bandwidth between trustedPC and itPC
*** Results: ['22.7 Gbits/sec', '22.7 Gbits/sec']
```

trustedPC and the IT LAN can communicate with each other, and hence the TCP connection goes through.

Important: If *iperf* takes longer than around 15 sec to run, this usually means the hosts can't reach each other; cancel with ctrl-c and include the result in your **screenshot.**

<u>Question 4 [30 pts]:</u> Verify Rule 3 with the command (`iperfudp 10M <host1> <host2>`). Are the individual results what you expect? Why? Include a **timestamped screenshot** and refer to it in your explanation after running `iperfudp` between these pairs of nodes:

- examServer and itWS

```
mininet> iperfudp 10M examServer itWS
*** Iperf: testing UDP bandwidth between examServer and itWS
*** Results: ['10M', '10.5 Mbits/sec', '10.5 Mbits/sec']
```

The university data center and IT department LAN can forward UDP packets to each other and hence the connection goes through.

- itWS and facultyWS

```
mininet> iperfudp 10M itWS facultyWS
*** Iperf: testing UDP bandwidth between itWS and facultyWS
*** Results: ['10M', '10.5 Mbits/sec', '10.5 Mbits/sec']
```

The faculty LAN and IT department LAN can forward UDP packets to each other and hence the connection goes through.

- facualtyWS, LabWS

```
mininet> iperfudp 10M facultyWS labWS
*** Iperf: testing UDP bandwidth between facultyWS and labWS
*** Results: ['10M', '10.5 Mbits/sec', '10.5 Mbits/sec']
```

The faculty LAN and student housing LAN can forward UDP packets to each other and hence the connection goes through.

- itWS and itPC

```
mininet> iperfudp 10M itWS itPC
*** Iperf: testing UDP bandwidth between itWS and itPC
*** Results: ['10M', '10.5 Mbits/sec', '10.5 Mbits/sec']
```

Devices on the same network can forward UDP packets to each other and hence the connection goes through

- trustedPC and itPC

```
mininet> iperfudp 10M trustedPC itPC
*** Iperf: testing UDP bandwidth between trustedPC and itPC
^C
Interrupt
```

trustedPC cannot forward UDP packets, so the connection does not go through. I waited for 5-ish minutes, but it wouldn't end, so I interrupted it.

- trustedPC and guestPC
  I used guest1 instead of guestPC

```
mininet> iperfudp 10M trustedPC guest1
*** Iperf: testing UDP bandwidth between trustedPC and guest1
^C
Interrupt
```

trustedPC cannot forward UDP packets, so the connection does not go through. I waited for 5-ish minutes, but it wouldn't end, so I interrupted it.

Important: A `UDP` test is considered successful if the `iperfudp` output displays two UDP bandwidth measurements between hosts. If one of the UDP bandwidth measurements is missing, it means that `UDP` traffic is not flowing. For example:

Pass: `*** Results: ['10M', '10.1 Mbits/sec', '10.1 Mbits/sec']`

Failed: `*** Results: ['10M', '', '10.0 Mbits/sec']`

Question [5 pts]: Extra Credit: Create a "discordServer" host on the Internet (with an public IP of your choice) that is only accessible from the Student LAN. Prove that students may access the server but the teaching staff cannot by including a **screenshot** of the following commands

- studentPC1 ping discordServer
- facultyPC ping discordServer
- iperfudp studentPC2 discordServer
- iperfudp facultyPC discordServer
- iperf studentPC2 discordServer
- iperf facultyPC discordServer